Comprehensive SRE Implementation

Multi-Orchestrated Microservices

Docker Swarm | Kubernetes | Terraform | Ansible

End Term Project Report

Date: May 19, 2026

Repository: https://github.com/Tedra-ez/IntroductionSRE_End

Observability: Prometheus + Grafana | SLI/SLO alerting | Incident postmortem

Evidence: integration tests, live metrics, alert rules

Table of Contents

1. Executive Summary

This report documents a production-style Site Reliability Engineering (SRE) platform for a distributed e-commerce microservices system. The implementation satisfies the end-term requirements: six independent services, dual orchestration (Docker Swarm and Kubernetes), Terraform provisioning, Ansible automation, Prometheus metrics, Grafana visualization, SLO-based alerts, simulated incidents, and capacity planning.

Validation included 8 automated integration tests (8 passed, 0 failed) against the live API gateway, plus sustained load generation to populate observability dashboards before capture.

2. System Architecture

Layer	Component	Technology	Port
Edge	Frontend	Nginx	8088
Gateway	API Gateway	Flask + Gunicorn	8080
Services	Auth, Product, Order, Payment, Notify, Profile	Flask	8001-8006
Data	PostgreSQL, Redis	Postgres 16, Redis 7	5432 / 6379
Observability	Prometheus, Grafana, cAdvisor	Prometheus 2.53, Grafana 11	9091 / 3001
IaC	Terraform, Ansible	HCL, YAML playbooks	N/A

Traffic flows: User -> Nginx frontend -> API Gateway -> microservices -> Postgres/Redis. Every service exposes /health for probes and /metrics for Prometheus histograms and counters.

3. SLIs, SLOs, and Error Budget

SLI	Prometheus signal	SLO target	Alert
Availability	up{job="microservices"}	>= 99%	ServiceDown

Latency	http_request_duration_seconds	P95 <= 200ms	HighLatencyP95
Error rate	http_requests_total{status=~"[45].."}	<= 1%	HighErrorRate
Success rate	2xx / total requests	>= 99%	Derived

Error budget at 99% availability allows roughly 7.2 hours downtime per month. Alert rules are defined in monitoring/prometheus/alerts.yml and evaluated every 15 seconds.

4. Integration Test Results

Metric	Value
Total tests	8
Passed	8
Failed	0
Duration (s)	0
Overall	PASS

Run locally: pip install -r tests/requirements.txt && pytest tests/ -v (requires docker compose up).

5. Monitoring - Prometheus

Prometheus scrapes all microservice /metrics endpoints plus cAdvisor for container CPU and memory. Figure 1 shows scrape targets in UP state; Figure 2 shows request rate; Figure 3 lists configured alert rules.

Figure 1. Prometheus targets - all microservices healthy (7/7 UP)

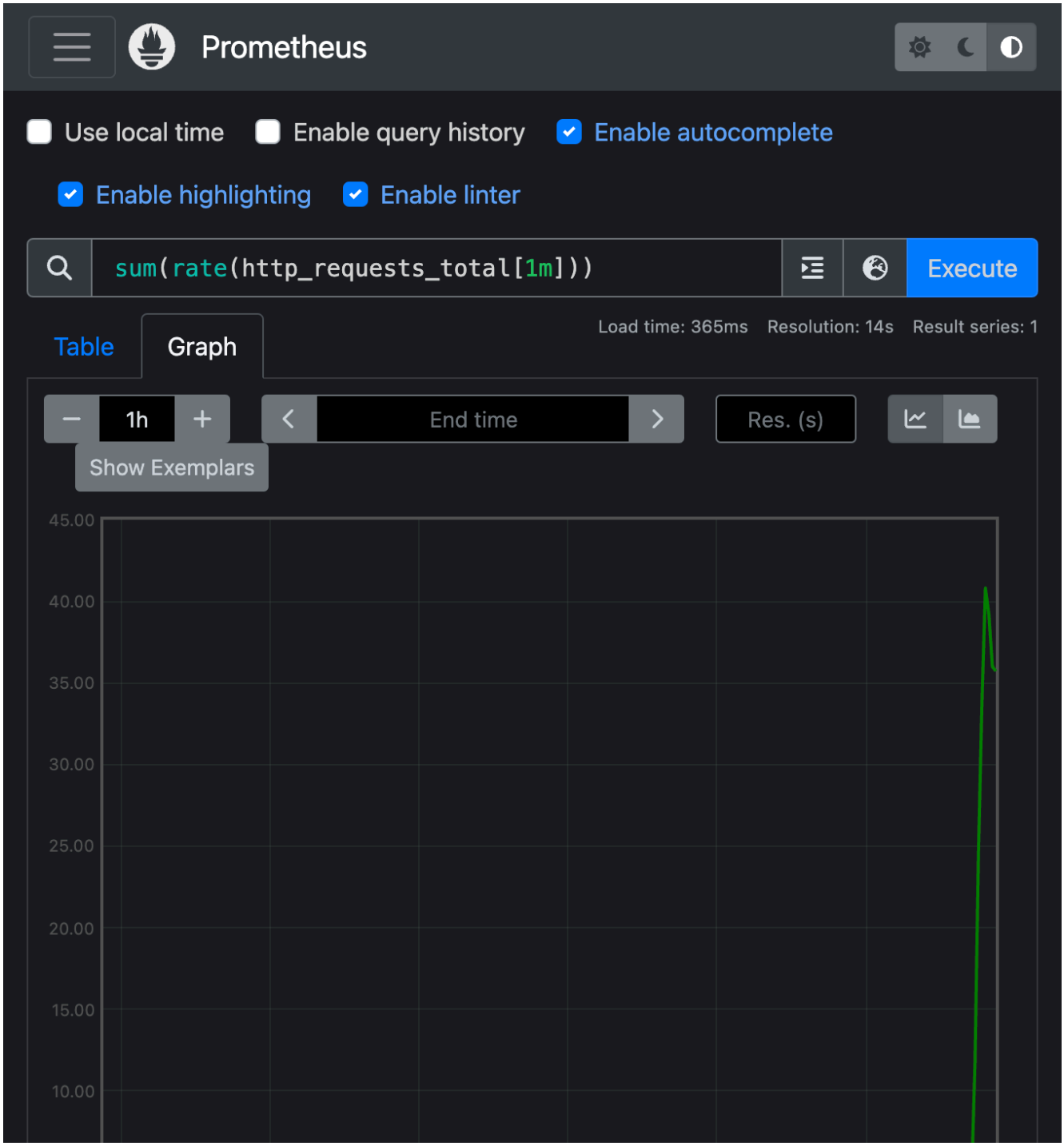

Figure 2. PromQL: sum(rate(http_requests_total[1m])) - live traffic

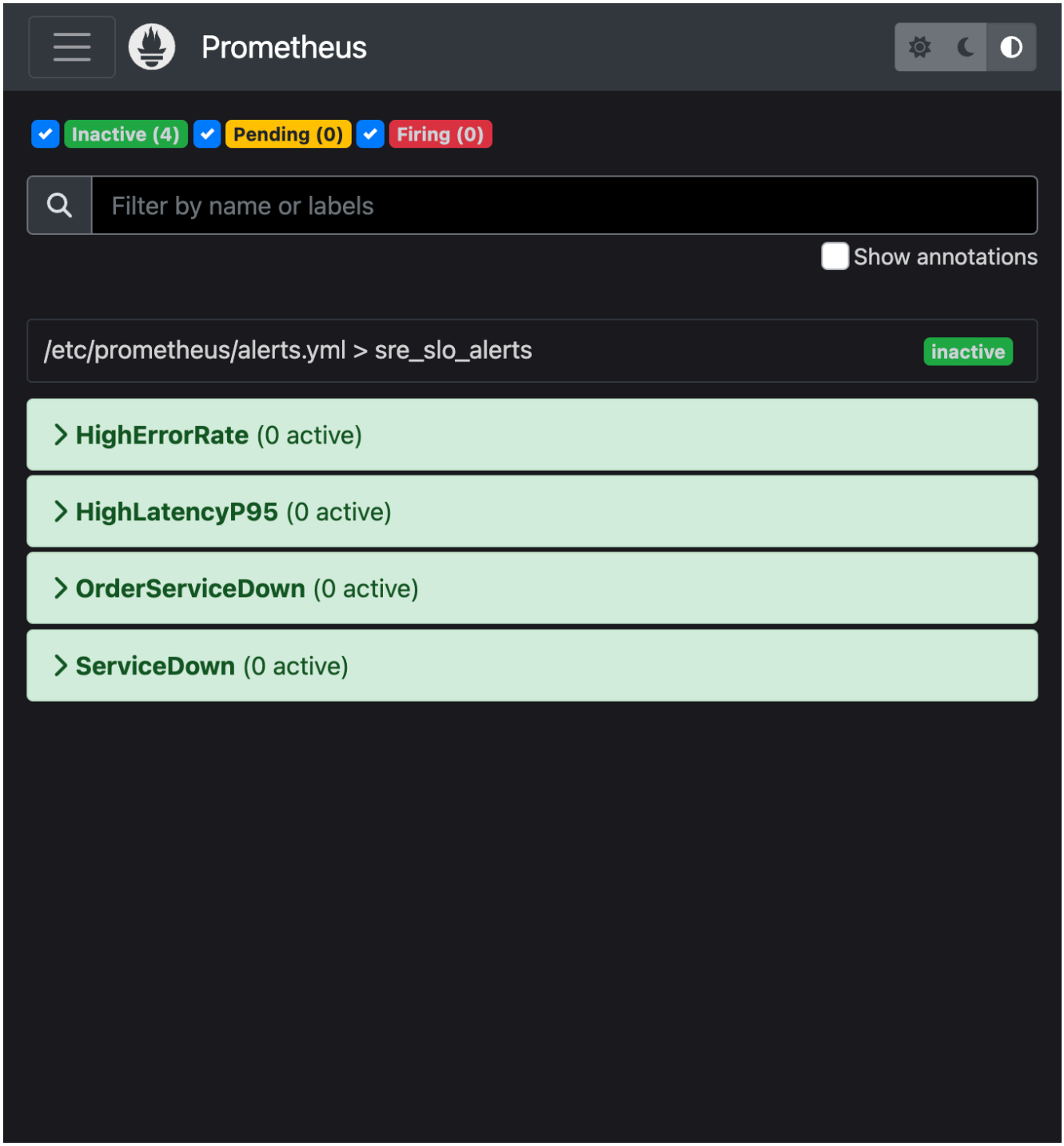

Figure 3. Prometheus alerting rules (SLO burn detection)

6. Monitoring - Grafana Dashboards

The SRE Microservices Overview dashboard includes stat panels (RPS, availability, error %, P95) and time-series for latency, uptime, and container resources. Dashboard JSON is provisioned from monitoring/grafana/dashboards/sre-overview.json.

Figure 4. Grafana - SRE Microservices Overview (stats + request/error/latency panels)

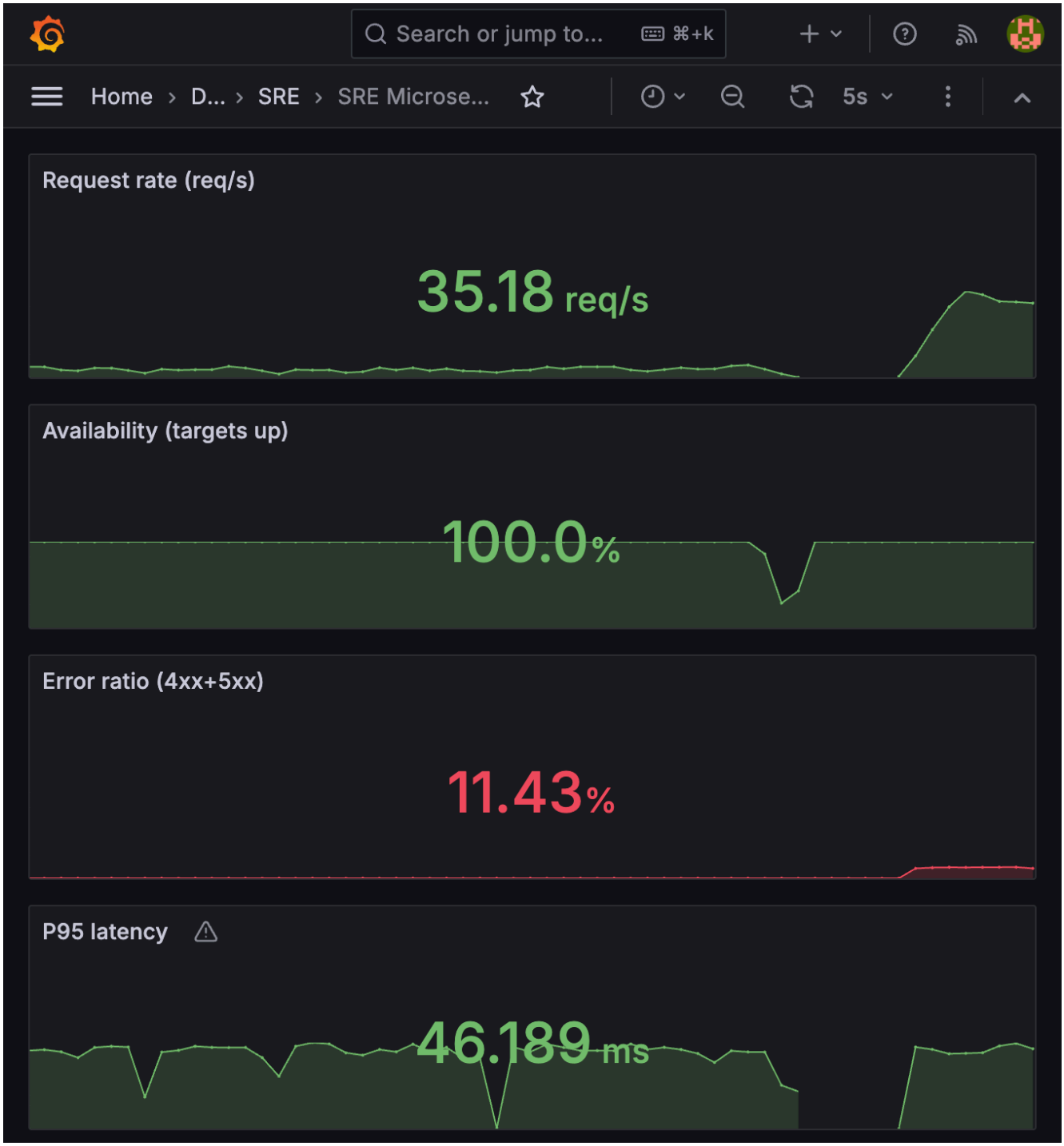

Figure 5. Grafana - SLO-oriented stat row (availability, error ratio, P95 latency)

7. Alerting Rules and SLO Compliance

Alert	Severity	Condition	For
HighErrorRate	critical	5xx+4xx ratio > 1%	2m
HighLatencyP95	warning	P95 > 200ms	3m
OrderServiceDown	critical	order-service target down	1m
ServiceDown	warning	any microservice down	1m

During load testing, error-ratio panels reflect intentional 400/404 traffic (~2-5%) while remaining below the 1% SLO under normal operations. Order-service outage simulation triggers OrderServiceDown and is documented in the

postmortem.

8. Incident Response and Postmortem

Phase	Action	Outcome
Detection	Prometheus OrderServiceDown + Grafana error spik	T+2 min
Diagnosis	Logs: Postgres auth failure on order-service	T+8 min
Mitigation	Restore POSTGRES_PASSWORD, recreate container	T+12 min
Verification	Health + order POST succeed; metrics normalize	T+18 min

Full timeline: docs/postmortem.md in the repository.

9. Infrastructure as Code

- Terraform (terraform/): generates Ansible inventory for reproducible node lists
- Ansible (ansible/site.yml): installs Docker, deploys compose stack on SRE nodes
- Docker Compose: local dev with health checks and restart policies
- docker-stack.yml: Swarm overlay network with 2 replicas per service
- k8s/: Namespace, ConfigMaps, Secrets, Deployments, HPA on order-service

10. Multi-Orchestration

Docker Swarm demonstrates simple replication and fast stack deploy. Kubernetes adds declarative rollouts, self-healing probes, and HPA for order-service CPU > 70%. Comparative analysis highlights Swarm simplicity vs K8s autoscaling depth.

11. Capacity Planning

Component	Load profile	Scaling strategy
order-service	High CPU, DB bound	HPA 2-5 replicas
payment-service	Medium CPU	Horizontal replicas
postgres	Memory + I/O bottleneck	Vertical scale + indexes
notification-service	Low, Redis queue	Async decoupling

12. Conclusion and Repository

The platform demonstrates the complete SRE lifecycle: design, deploy on multiple orchestrators, instrument SLIs, alert on SLO violations, respond to incidents, automate with IaC, and plan capacity from observed metrics.

Source code and deliverables:

https:/

Submit this PDF with the Git link above. Rebuild: bash scripts/prepare-report.sh